

Relatório do Projeto 1

Link do GitHub:

<https://github.com/jhapreis/7600017-introducao-fisica-computacional/tree/main/projeto1>

Sumário

1	Tarefa 1	1
1.1	Cálculo do volume do toro dados os raios interno e externo	1
2	Tarefa 2	3
2.1	Cálculo da área e do volume de um paralelepípedo de base triangular	3
3	Tarefa 3	6
3.1	Busca e ordenação dos menores M valores em uma lista de tamanho N	6
4	Tarefa 4	8
4.1	Erro de precisão no cálculo com real^*4	8
4.2	Erro de precisão no cálculo com real^*8	9
5	Tarefa 5	12
6	Tarefa 6	13
6.1	Programa para o cálculo do volume da esfera em d dimensões	13
6.2	Gráfico comparativo do volume com raio variando	14
7	Tarefa 7	15
7.1	Razão entre volume do cubo e da esfera em d dimensões	15
7.2	Estimativas do Número de Avogrado em d dimensões	15

1 Tarefa 1

1.1 Cálculo do volume do toro dados os raios interno e externo

No toro, consideremos¹ r_a como o raio do maior círculo e r_b como o raio do menor círculo.

A área superficial A é dada por

$$A = 4\pi^2 r_a r_b$$

O volume V é dado por

$$V = 2\pi^2 r_a r_b^2$$

E a relação de r_a e r_b com os raios interno r_1 e externo r_2 é dada por

$$r_a = \frac{r_1 + r_2}{2}$$
$$r_b = \frac{r_2 - r_1}{2}$$

Para o programa do FORTRAN, as funções auxiliares e o código principal para exibir os valores (dado r_1 e r_2) são mostrados nas Figuras 1 e 2, respectivamente.

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
! Funções
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

! Função para o cálculo do raio R do toro
double precision function radius_a(r1, r2)
double precision r1, r2
radius_a = 0.5d0 * (r1 + r2)
return
end

! Função para o cálculo do raio r do toro
double precision function radius_b(r1, r2)
double precision r1, r2
radius_b = 0.5d0 * (r2 - r1)
return
end

! Função para o cálculo da Área A
double precision function area(r1, r2)
double precision radius_a, radius_b ! outras funções
double precision r1, r2, pi ! variáveis
pi = acos(-1d0)
area = 4d0 * pi**2 * radius_a(r1, r2) * radius_b(r1,r2)
return
end

! Função para o cálculo do Volume V
double precision function volume(r1, r2)
double precision radius_a, radius_b ! outras funções
double precision r1, r2, pi ! variáveis
pi = acos(-1d0)
volume = 4d0 * pi**2 * radius_a(r1, r2) * radius_b(r1,r2)**2
return
end
```

Figura 1: Funções auxiliares para o cálculo da área e do volume do toro.

¹Como o FORTRAN não é case-sensitive, as variáveis tiveram seus nomes alterados. Por isso, $R = r_a$ e $r = r_b$

```

! Cálculo da Área e do Volume do Toróide
! Definições
!   Ra: raio até o meio do toróide
!   Rb: raio da circunferência da espessura do toróide
!   r1: raio interno do toróide
!   r2: raio externo do toróide
! Cálculo:
!   A = 4 * pi**2 * Ra * Rb
!   V = 4 * pi**2 * Ra * Rb**2
!   R = (r2 + r1) / 2
!   r = (r2 - r1) / 2

! Ler o valor de input
double precision r1, r2
double precision radius_a, radius_b, area, volume
read (*,*) r1,r2

print *, area(r1,r2), volume(r1,r2)
end

```

Figura 2: Código principal para exibir a área e o volume do toro.

2 Tarefa 2

2.1 Cálculo da área e do volume de um paralelepípedo de base triangular

Para o cálculo do volume V do paralelepípedo de base triangular, podemos usar que

$$\begin{aligned} V = V_{triangular} &= \frac{1}{2} V_{retangular} \\ &= \frac{1}{2} (|v_1 \times v_2| \cdot v_3) \end{aligned}$$

Já para o cálculo da área lateral:

$$\begin{aligned} A &= \sum (areas\ das\ faces) \\ &= |u \times w| + |v \times w| + |(u - v) \times w| \end{aligned}$$

Além disso, para garantir que os vetores constituem um sólido, é necessário garantir que

1. v_1 e v_2 não são paralelos, ou seja:

$$|v_1 \times v_2| > 0$$

2. se v_3 não pertence ao mesmo plano formado por v_1 e v_2 :

$$|(v_1 \times v_2) \cdot v_3| > 0$$

Podemos ver as definições das funções incorporadas no programa², conforme as Figuras 3, 4 e 5.

²Observe que no programa, para simplicidade, as variáveis foram renomeadas: $v1 = u$, $v2 = v$ e $v3 = w$.


```

implicit none

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
! Num paralelepípedo de base triangular, programa para o cálculo
!   do volume
!   da sua área lateral
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

double precision u(3), v(3), w(3)

double precision norm, dot_prod

double precision uv(3), norm_uv, u_minus_v(3)

double precision uw(3), vw(3), u_minus_v_w(3)

double precision prod_misto

double precision volume, area

double precision zero
zero = 1.0d-12 ! considerando ~ 0, para desconsiderar erros de ponto flutuante

print *, 'Digite as coordenadas de v1:'
read(*,*) u(1),u(2),u(3)

print *, 'Digite as coordenadas de v2:'
read(*,*) v(1),v(2),v(3)

print *, 'Digite as coordenadas de v3:'
read(*,*) w(1),w(2),w(3)

```

Figura 4: Inicialização das variáveis do programa.

```

call cross_prod(u, v, uv)

! Verifica se v1 e v2 são paralelos: se o módulo do vetor normal é nulo
norm_uv = norm(uv)
if(norm_uv.lt.zero) then
    print *, 'ERROR: v1 e v2 sao paralelos'
    stop
end if

! Verifica se o vetor v3 não é coplanar com v1 e v2: se v3 é perpendicular ao vetor normal ao plano de v1 e v2 (ou seja, está no me
prod_misto = dot_prod(w, uv)
if( abs(prod_misto).lt.zero ) then
    print *, 'ERROR: v3 e coplanar com v1 e v2'
    stop
end if

! Cálculo do volume:  $V = 1/2 * w \cdot /u \times v/ = 1/2 * prod\_misto$ 
volume = 0.5d0 * abs(prod_misto)

! Cálculo da área lateral:  $A = /u \times w/ + /v \times w/ + /(u - v) \times w/$ 
call cross_prod(u, w, uw)
call cross_prod(v, w, vw)
call vector_diff(u, v, u_minus_v)
call cross_prod(u_minus_v, w, u_minus_v_w)

area = norm(uw) + norm(vw) + norm(u_minus_v_w)

! Fim do programa
print *, 'Volume =', volume
print *, 'Area   =', area
stop
end

```

Figura 5: Código principal para execução do programa, com as validações dos vetores.

3 Tarefa 3

3.1 Busca e ordenação dos menores M valores em uma lista de tamanho N

Dado uma lista de tamanho N , o programa busca os M menores valores, de maneira ordenada, e os imprime no terminal.

Definição das variáveis:

- vl_{row} : o valor da linha atual (lido do arquivo de input)
- vl_{min_last} : o menor valor encontrado em todos os laços do loop
- vl_{min} : o menor valor encontrado no laço atual (que ainda não foi incluído na lista)

A estratégia principal consiste em

1. Fazer dois loops
 - O primeiro em M , para buscar o total de M registros (ou seja, não é necessário continuar o loop após achar o que precisamos; não é necessário ordenar a lista inteira)
 - O segundo em N , procurando em cada elemento da lista um candidato
2. Definir os valores de inicialização das variáveis
 - $vl_{min_last} = -\infty$ (uma única vez)
 - $vl_{min} = +\infty$ (resetando a cada laço do loop)
3. Ao fim de cada laço em M , salvar o menor valor achado no loop em N

Vide o código nas Figuras 6 e 7.

```
implicit none

! Definição dos números n (o tamanho do arquivo de input) e m (o subconjunto de menores valores)
integer n, m

! Definição de variáveis auxiliares para loop dentro do arquivo
integer i, j

! Valores da linha do arquivo, do atual candidato e do último salvo até então
double precision vl_row, vl_min, vl_min_last

! Valor máximo, para comparação. Uma constante, interpretada como infinito
double precision inf

inf = 1.0d300

print *, "Insira o tamanho do arquivo original: "
read (*,*) n

print *, "Insira o tamanho do subconjunto de menores valores: "
read (*,*) m

if( (n.lt.0).or.(m.lt.0) ) then
    print *, 'ERROR: (n,n) devem ser não-negativos: ', m, n
    stop
else if(n.lt.m) then
    print *, 'ERROR: m não pode ser maior que n', m, n
    stop
end if

! Abrir arquivo de saída e salvar o valor de m
open(unit=99, file="tarefa-3-saida-1.dat")

write(99, *) m

! Para iniciar o loop, consideramos que o menor já valor encontrado é -inf
vl_min_last = -inf
```

Figura 6: Preparação para os loops, com a inicialização das variáveis e definição das constantes.


```

do i=1,m

    ! Start/reset do vl_min do loop
    vl_min = inf

    ! Ler o arquivo de dados de input
    open(unit=1, file="tarefa-3-entrada-1.in")

    do j=1,n

        read(1, *) vl_row

        if(j.eq.1) then
            vl_min = vl_row
        else
            if( (vl_row.lt.vl_min).and.(vl_row.gt.vl_min_last) ) then
                vl_min = vl_row
            end if

        end if

        ! print *, 'j=', j, 'vl_min=', vl_min, 'vl_min_last=', vl_min_last

    end do

    ! print *, 'i=', i, 'vl_min=', vl_min

    write(99, *) vl_min

    ! Fecha o arquivo, para ler novamente do começo no próximo laço
    close(1)

    ! Define o menor valor como sendo o último mínimo encontrado
    vl_min_last = vl_min

end do

close(99)

end

```

Figura 7: Execução dos loops.

O programa principal recebe um valor real de input (de preferência³, um valor entre $[-\pi, \pi]$.) e exibe os valores finais associados ao valor da série (definida como função) e também o valor da função $\cos(x)$ nativa do FORTRAN.

```

real x, cos_series, fx, cos_x, error

read(*,*) x

fx = cos_series(x)

cos_x = cos(x)

error = abs(cos_x - fx)

print *, 'x=', x, 'fx=', fx, 'cosx=', cos_x, 'err=', error

stop
end

```

Figura 10: Código principal para a comparação dos valores da série, como REAL*4.

4.2 Erro de precisão no cálculo com real*8

Modificando o programa anterior para o tipo real*8 (ou double precision), temos as seguintes funções definidas nas Figuras 11 e 12.

```

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
! Função para o cálculo do fatorial de um inteiro
! Utiliza variáveis reais apenas para computar com maior precisão
!
! Parâmetros:
!   n: número inteiro (como double precision). Exemplo: 2.0d0, 12.0d0
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
double precision function factorial(n)
  double precision n, i
  factorial = 2.0d0
  i = n
  do while (i.ge.3.0d0)
    factorial = factorial*i
    i = i - 1.0d0
  end do
  return
end

```

Figura 11: Função para o cálculo do fatorial de um inteiro, com tipo REAL*8.

³Isso porque valores especialmente muito grandes ou pequenos podem causar overflow/underflow na computação da série

5 Tarefa 5

6.1 Programa para o cálculo do volume da esfera em d dimensões

$$V_d = \frac{\pi^{d/2}}{\Gamma(1 + d/2)} R^d$$
$$\Gamma(1/2) = \sqrt{\pi}$$

$$\Gamma(1) = 1$$

$$\Gamma(x+1) = x\Gamma(x)$$

$$\begin{aligned} d : par &\longrightarrow \Gamma_{inicial} = \Gamma(1) = 1 \\ d : impar &\longrightarrow \Gamma_{inicial} = \Gamma(1/2) = \sqrt{\pi} \end{aligned}$$

Figura 15: Função para o cálculo de Γ dado o número de dimensões

O programa principal espera que sejam passadas duas variáveis de input (r, d) e salva os dados de ($r, d, volume(d)$) no arquivo *arefa6.dat*. Vide Figura 17.

7 Tarefa 7

7.1 Razão entre volume do cubo e da esfera em d dimensões

Da tarefa anterior, temos que

$$V_{esfera} = \frac{\pi^{d/2}}{\Gamma(1 + d/2)} R^d$$

Além disso, também temos que

$$V_{cubo} = 1^d = 1$$

Portanto, a razão V_{cubo}/V_{esfera} é

$$\frac{V_{cubo}}{V_{esfera}} = \frac{\Gamma(1 + d/2)}{\pi^{d/2} R^d}$$

Olhando para o gráfico da Figura 18, podemos observar que para valores de $d > 12$, devido ao colapso da função Γ , o volume da esfera colapsa e passamos a ter uma razão crescente com $V_{cubo}/V_{esfera} > 1$.

Na verdade, observando que

$$\lim_{d \rightarrow \infty} V_{esfera} = 0$$

temos que

$$\lim_{d \rightarrow \infty} \frac{V_{cubo}}{V_{esfera}} = \infty$$

7.2 Estimativas do Número de Avogrado em d dimensões

Considerando que o número de Avogrado N_A pode ser estimado como

$$N_A \sim \frac{V_{macro}}{V_{atomo}} = \frac{(10^{-3})^d}{(10^{-10})^d} = 10^{7d}$$

para o nosso mundo tridimensional, teríamos $N_A \sim 10^{21}$.

Também é possível fazer outra consideração: se considerarmos que as medidas fornecidas são razoáveis com a dimensão do nosso mundo tridimensional, então podemos considerar que

$$d = 3 \longrightarrow N_A \sim 10^{23} = 10^{23/3} \approx 10^{7d}$$